BombLab 实验报告

课程：计算机系统基础（ICS）Lab2 · 二进制炸弹实验

姓名：严泳垚	学号：24307090048
学期：2026 秋季	GitHub 仓库：https://github.com/Yyylanli/ics-2026-bomblab

一、实验概述

本实验需要逆向一个 64 位 ELF 二进制程序 bomb。它包含 6 个正式关卡（phase_01~phase_06）和 1 个必做的隐藏关卡（Honor，secret_*）。每一关运行时都会接收一个 64 位 challenge，由关卡内部函数计算出对应口令；只有拆除程序 defuser.c 针对任意 challenge 都能算出正确口令时，该关才算通过。

由于公开测试和最终评分分别使用 10 个和 128 个不同的随机 seed，背下某一次的口令数字无法得分，必须把汇编中隐含的通用计算关系完整还原成 C 代码。

核心方法：用 objdump -d bomb > bomb.asm 反汇编；定位各关卡与核心计算函数；逐条翻译数据传送、算术、位运算、条件跳转、内存访问与（间接）调用；在 defuser.c 中用 32 位无符号运算实现；用 python3 public_test.py 验证。所有运算按 2^32 自然回绕。

二、实验环境

操作系统：Windows 11，通过 WSL2 使用 Ubuntu 22.04；编辑器：Visual Studio Code（Remote-WSL）；工具链：gcc、make、objdump、gdb、Python 3；版本控制：Git + GitHub。

三、关卡主函数的通用结构

每个正式关卡主函数结构一致：保存 challenge（rsi）；调用 valid_u32_fields 检查输入字段数；调用 sscanf 读入若干 uint32；调用本关核心计算函数；用 cmp 把结果与输入逐一比较，全等才通过。参数寄存器依次为 rdi、rsi、rdx、rcx、r8、r9，32 位结果存于 %eax。

四、各关卡分析

第 1 关：函数调用

关键汇编特征：主函数依次三条 call 调用 p1_step0/1/2，参数按 SysV 约定经 rdi/rsi/rdx 传递、结果经 %eax 返回；随后 add 合并、最后一条 xor $0x88f 即为最终异或常量。

知识点：函数调用约定、参数寄存器、返回值。核心函数 p1_compute 顺序调用 p1_step0/1/2。令 a=challenge>>32，b=challenge>>16，d=challenge 低 32 位：

```
s1 = (b + 0x9a93) ^ a
s0 = 3*(d - a) + 0xcad6
answer = (s0 + 2*s1) ^ 0x88f
```

只需 1 个数字 answer。

第 2 关：数组 / 循环 / 递推

关键汇编特征：一个 for 循环，计数器 %ecx 从 1 加到 5；循环体内用 shr 取 challenge 不同位段、and $0xf 掩位、xor 与 imul $5 做递推；数组元素按 uint32 步进 4 访问（(%rsi,%rcx,4)）。

知识点：循环、数组、递推。共 6 个数字：

```
values[0] = 3 * ((challenge >> 36) & 0x7f)
for i = 1..5:
    values[i] = 5*values[i-1] ^ (((challenge >> (i+13)) & 0xf) + i)
```

第 3 关：switch / 跳转表

关键汇编特征：cmp $8,%eax 后 ja 越界处理，核心是 jmpq *跳转表(,%rax,8) 间接跳转；8 个分支块分别做 add/imul/xor/shr 组合，即 switch 的 8 个 case。

知识点：多分支、跳转表（基址 0x405820）。输入 2 个数字：

```
k   = challenge & 7
arg = (challenge >> 3) ^ (challenge >> 32)
values[0] = k
values[1] 按 k 分支：
  k=0: 5*arg+0xaed0      k=1: (arg^0x1dee)+(arg>>1)
  k=2: (arg^0xa9cf)+(arg>>2)  k=3: 5*arg+0x2422a
  k=4: 5*arg+0x1f58b     k=5: (2*arg)^0xdedd
  k=6: 5*arg+0x50e1      k=7: (4*arg)^0xb524
```

第 4 关：递归

关键汇编特征：p4_walk 内部 cmp %rdi,%r12 比较 t 与 mid，je 命中、jl/jg 两个分支都再次 call p4_walk（递归自调用），返回后 imul $5、xor、再 xor 分支常量 0x13/0x18。

知识点：递归、二分思想。核心 p4_walk(t,lo,hi,key)，t=challenge&0x7f，key=(challenge>>7)^(challenge>>32)，初值 lo=0、hi=0x7f，mid=lo+((hi-lo)>>1)：

```
mid==t : (mid + 0x3b) ^ key
t > mid : (5*walk(t,mid+1,hi,key) ^ (mid+key)) ^ 0x13
t < mid : (5*walk(t,lo,mid-1,key) ^ (mid+key)) ^ 0x18
```

第 5 关：结构体 / 链表

关键汇编特征：结构体三字段分别在偏移 0（next 指针，8 字节）、8（b，uint32）、0xc（c，uint32）；循环中 mov (%rbx),%rbx 沿 next 前进，mov 0x8(%rbx)、mov 0xc(%rbx) 取字段。

知识点：结构体布局、指针、链表遍历。数据段 12 个节点，每个为 {uint64 next; uint32 b; uint32 c;}；8 项起始表索引为 challenge&7。算法：

```
sel   = challenge & 7
steps = (challenge >> 3) & 3      // 沿链表走 0~3 步
cur   = start[sel]
x     = ((challenge>>6) ^ (challenge>>32) ^ c[cur] ^ 0x55c) + b[cur]
for i = 1..steps:
    cur = nxt[cur]
    x   = (x ^ c[cur]) + b[cur]
values[0] = x            // 注意顺序：第一个输入比对返回值 x
values[1] = 最后节点 b
```

调试中曾因 sscanf 输出指针与 cmp 比较顺序写反而全错，核对后修正为 values[0]=x、values[1]=b。

第 6 关：函数指针表 / 间接调用

关键汇编特征：两次 mov 从表 0x405e00 取出函数指针到寄存器，再 call *%rax 间接调用（非直接 call 标签）；循环移位由 shl/shr 两条指令拼出 rol/ror。

知识点：函数指针、间接调用 call *%reg、循环移位。表 0x405e00 共 8 项：{op4,op1,op5,op2,op0,op5,op3,op0}。六个运算函数：

```
op0(x) = (x ^ (x>>5)) + 0xa7d4
op1(x) = ror32(x, 7)
op2(x) = rol32(x, 2) ^ 0x609
op3(x) = (x ^ 0x6872) + 8*x
op4(x) = (x + 0x2e19) ^ (x>>1)
op5(x) = ((x<<7) + 0x1a84) ^ (5*x)
```

C 无原生循环移位且禁止内联汇编，用 (x<<n)|(x>>(32-n)) 实现。dispatch 连续两次查表调用：

```
arg    = (challenge >> 6) ^ (challenge >> 32)
ra     = table[challenge & 7](arg)
r1     = ra ^ 0xf91f
rb     = table[(challenge>>3) & 7](r1)
answer = rb + 0xdb29
```

五、Honor 隐藏关

关键汇编特征：secret_gate 先 call secret_copy 复制输入，随后 cmp 与字符串 "OPEN" 逐字节比较（偏移 32 处），再 call secret_phase 读 3 个数；secret_phase 内 call transcript_context 取得 context，mix 函数由 rol（shl/shr 拼出）、add、xor 构成。

入口 secret_gate 把输入字符串复制到缓冲区（secret_copy 等价 strcpy），要求：长度满足 strlen-36 在 0~127；缓冲区偏移 32 处必须为常量 “OPEN”；偏移 36 起由 secret_phase 读 3 个无符号整数。transcript_context() 由前 6 关答案算出 context，phase_challenge(...,7) 为本关 challenge，二者经接口直接传入。

辅助函数 secret_mix(x) = ((rol32(x,7) + 0xd6dd) ^ 0x55b9)。三个数字（32 位）：

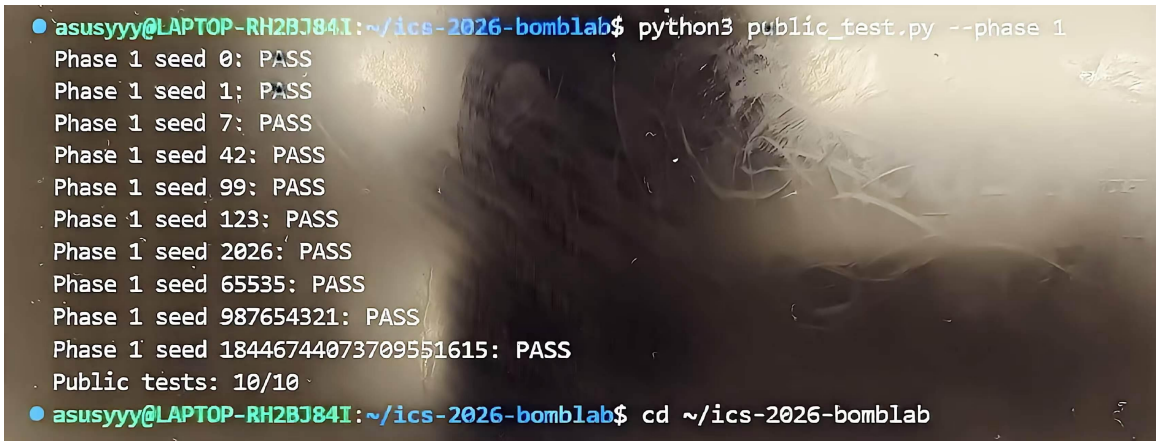
```
v0 = context & 3
A  = (challenge>>32) ^ challenge
B  = (context>>32)  ^ context
v1 = mix(A^B) ^ ((v0*0xd6dd) ^ (context>>16))
v2 = v1 ^ (B + v0 + 0x55b9 + (A>>7))
```

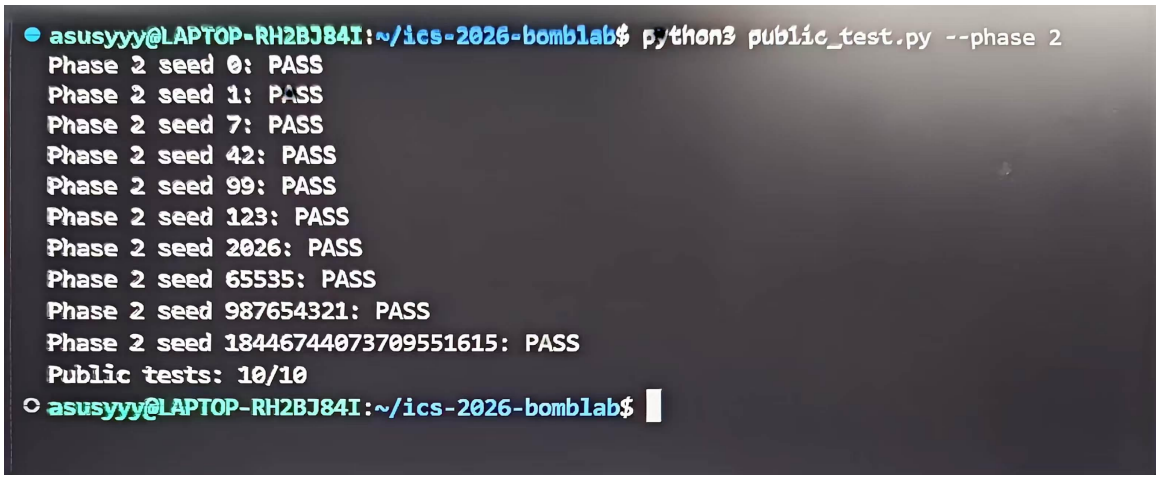
输出一行 ASCII：前 32 个填充字符 + “OPEN” + 空格分隔的 v0 v1 v2。

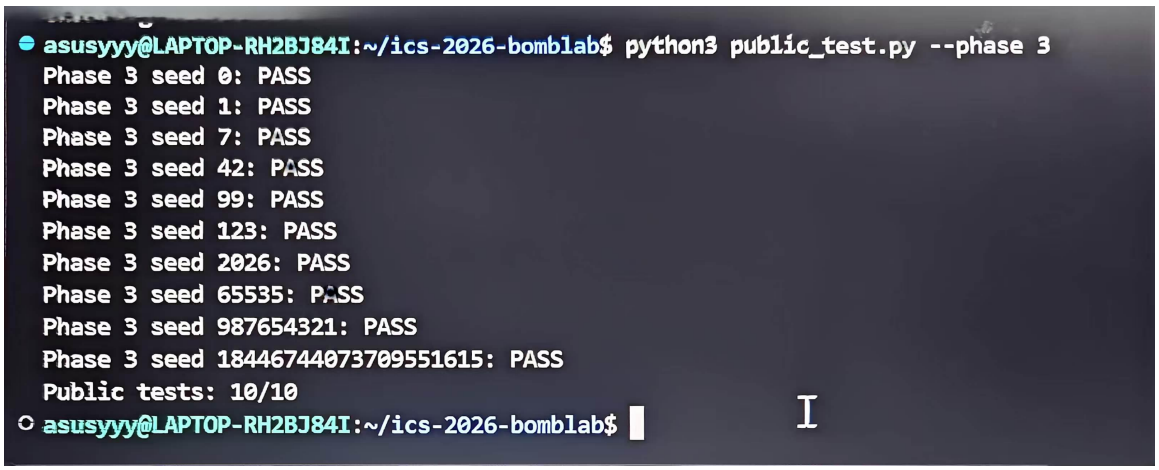
六、验证结果

6.1 公开 seed 测试（tests/public.json 的 10 个 seed）

使用 python3 public_test.py：Phase 1~6 各 10/10 PASS（合计 60/60），Secret 10/10 PASS。即 Public tests: 60/60，Secret public tests: 10/10。结果图如下：


asusyyy@LAPTOP-RH2BJ84I:~/ics-2026-bomblab$ python3 public_test.py --phase 1
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
Phase 1 seed 18446744073709551615: PASS
Public tests: 10/10
asusyyy@LAPTOP-RH2BJ84I:~/ics-2026-bomblab$ cd ~/ics-2026-bomblab

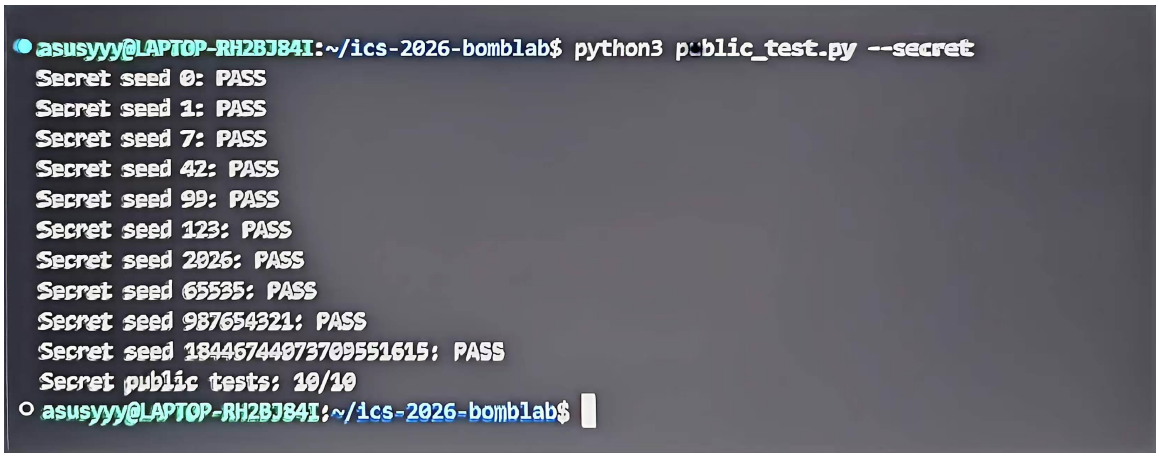

asusyyy@LAPTOP-RH2BJ84I:~/ics-2026-bomblab$ python3 public_test.py --phase 2
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
Public tests: 10/10
asusyyy@LAPTOP-RH2BJ84I:~/ics-2026-bomblab$


asusyyy@LAPTOP-RH2BJ84I:~/ics-2026-bomblab$ python3 public_test.py --phase 3
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Public tests: 10/10
asusyyy@LAPTOP-RH2BJ84I:~/ics-2026-bomblab$

```
asusyyy@LAPTOP-RH2BJ84I:~/ics-2026-bomblab$ python3 public_test.py --phase 4
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
Public tests: 10/10
asusyyy@LAPTOP-RH2BJ84I:~/ics-2026-bomblab$
```

```
asusyyy@LAPTOP-RH2BJ84I:~/ics-2026-bomblab$ python3 public_test.py --phase 5
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
Public tests: 10/10
asusyyy@LAPTOP-RH2BJ84I:~/ics-2026-bomblab$
```

```
asusyyy@LAPTOP-RH2BJ84I:~/ics-2026-bomblab$ python3 public_test.py --phase 6
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 10/10
asusyyy@LAPTOP-RH2BJ84I:~/ics-2026-bomblab$
```



6.2 公开列表之外的 seed 验证（报告要求第 2 条）

另取公开列表之外的 seed 12345，对每个已完成关卡（含 Honor）验证。正式 6 关逐关执行：

```
./defuser --seed 12345 --phase N | ./bomb --seed 12345 --phase N   (N=1..6)
```

各关结果如下表（每关均使用公开列表之外的 seed 12345）：

关卡	seed 12345 验证结果
Phase 1	Phase 1 defused.
Phase 2	Phase 2 defused.
Phase 3	Phase 3 defused.
Phase 4	Phase 4 defused.
Phase 5	Phase 5 defused.
Phase 6	Phase 6 defused.
Honor (secret)	Secret phase defused.

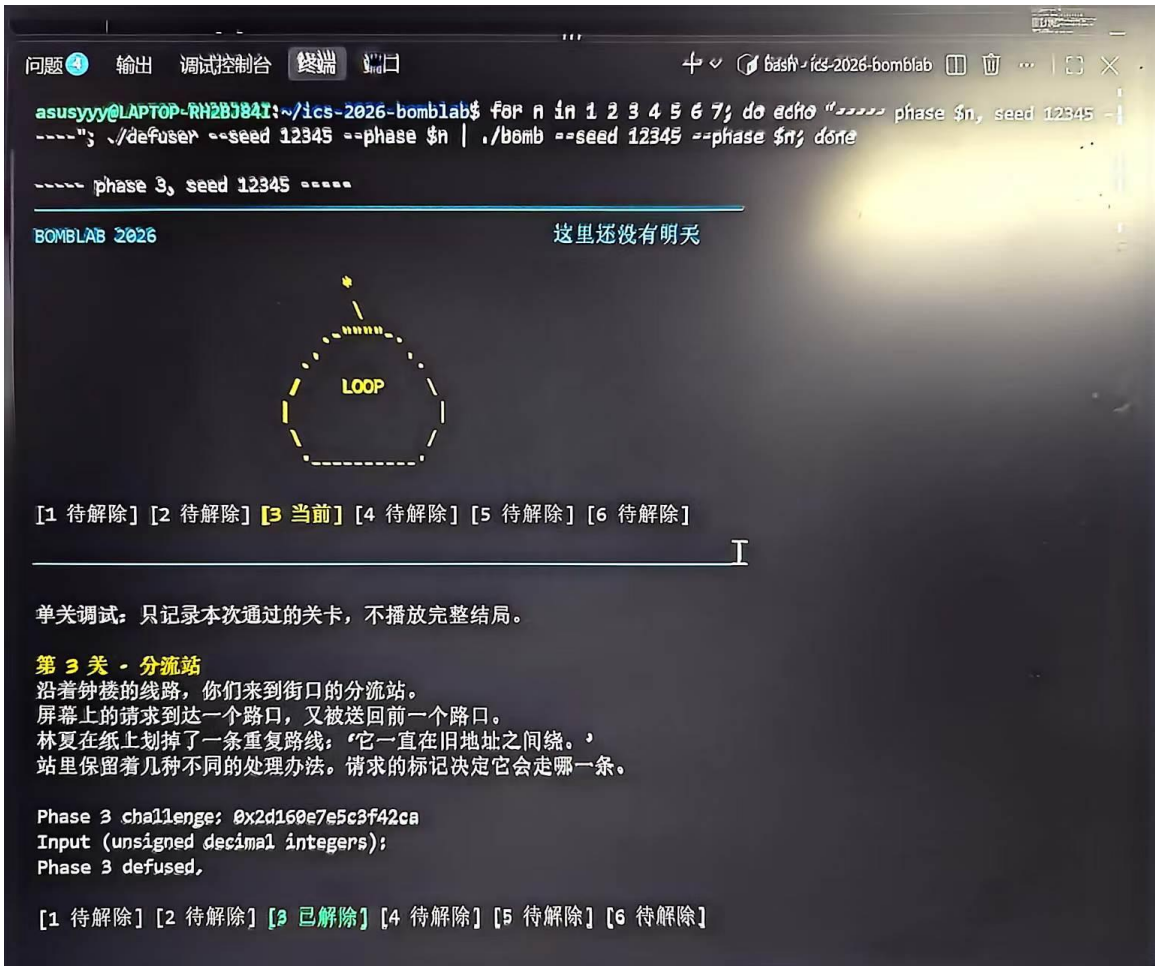
Honor 需把前 6 关答案与第 7 行秘密文本拼接后喂给普通模式 bomb：

```
{ ./defuser --seed 12345; ./defuser --seed 12345 --secret; } \
  | ./bomb --seed 12345 | grep -E "defused"
```

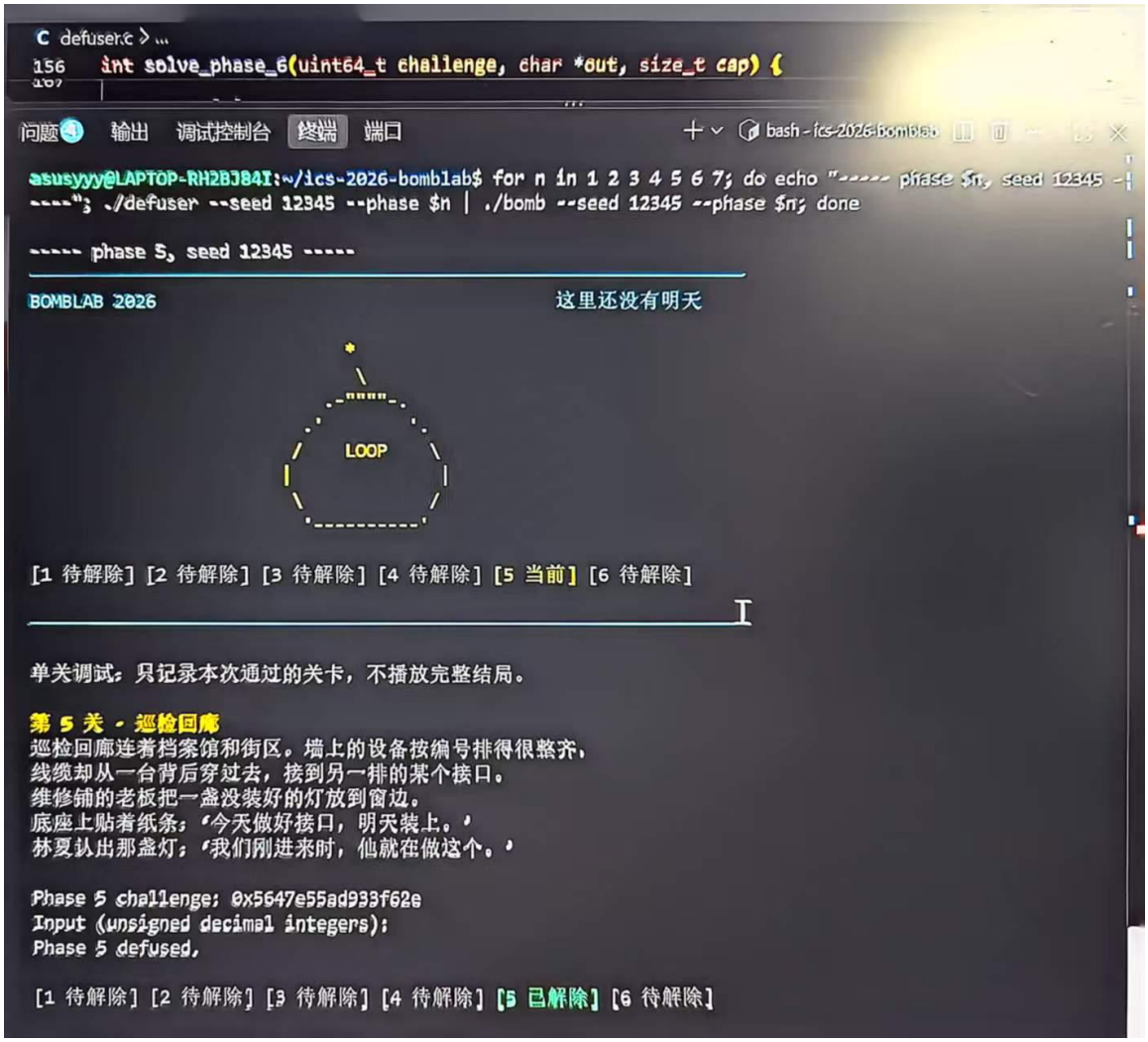
最终输出：Phase 1~6 defused、Congratulations! You've defused the bomb.、Secret phase defused.，全部通过。结果图如下：


asusyyy@LAPTOP-RH2BJ84I:~/ics-2026-bomblab$ for n in 1 2 3 4 5 6 7; do echo "----- phase $n, seed 12345 -----"; ./defuser --seed 12345 --phase $n | ./bomb --seed 12345 --phase $n; done
----- phase 1, seed 12345 -----
BOMBLAB 2026
这里还没有明天
LOOP
[1 当前] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
单关调试：只记录本次通过的关卡，不播放完整结局。
第 1 关 - 维护站
终端上留着一份维护记录。
你按了一下回车，屏幕亮起，六个指示灯中只有第一个在闪。
林夏把旁边的椅子拖过来：“我看记录，你看程序。”
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。
Phase 1 challenge: 0x346edce5f713f8ed
Input (unsigned decimal integers):
Phase 1 defused.
[1 已解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

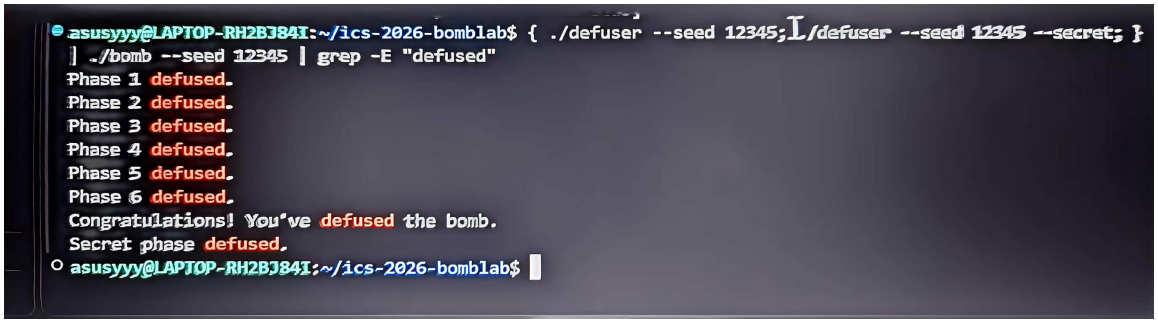

defuser.c > ...
156 int solve_phase_6(uint64_t challenge, char *out, size_t cap) {
问题 输出 调试控制台 终端 端口
bash - ics-2026-bomblab
asusyyy@LAPTOP-RH2BJB4I:~/ics-2026-bomblab$ for n in 1 2 3 4 5 6 7; do echo "
./defuser --seed 12345 --phase $n | ./bomb --seed 12345 --phase $n; done
phase 2, seed 12345
BOMBLAB 2026
这里还没有明天
LOOP
[1 待解除] [2 当前] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
单关调试：只记录本次通过的关卡，不播放完整结局。
第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’
Phase 2 challenge: 0x1e9a57bc80e6721d
Input (unsigned decimal integers):
Phase 2 defused.
[1 待解除] [2 已解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]


问题 4
输出
调试控制台
终端
端口
bash - ics-2026-bomblab
asusyyy@LAPTOP-RH2BJ84I:~/ics-2026-bomblab$ for n in 1 2 3 4 5 6 7; do echo "----- phase $n, seed 12345 -----"; ./defuser --seed 12345 --phase $n | ./bomb --seed 12345 --phase $n; done
----- phase 3, seed 12345 -----
BOMBLAB 2026
这里还没有明天
LOOP
[1 待解除] [2 待解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]
单关调试：只记录本次通过的关卡，不播放完整结局。
第 3 关 - 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：“它一直在旧地址之间绕。”
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。
Phase 3 challenge: 0x2d160e7e5c3f42ca
Input (unsigned decimal integers):
Phase 3 defused.
[1 待解除] [2 待解除] [3 已解除] [4 待解除] [5 待解除] [6 待解除]


C defuser.c > ...
156 int solve_phase_6(uint64_t challenge, char *out, size_t cap) {
问题 输出 调试控制台 终端 端口
bash - ics-2026-bomblab
asusyyy@LAPTOP-RH2BJ84I:~/ics-2026-bomblab$ for n in 1 2 3 4 5 6 7; do echo "-----"; ./defuser --seed 12345 --phase $n | ./bomb --seed 12345 --phase $n; done
----- phase 4, seed 12345 -----
BOMBLAB 2026
这里还没有明天
LOOP
[1 待解除] [2 待解除] [3 待解除] [4 当前] [5 待解除] [6 待解除]
单关调试：只记录本次通过的关卡，不播放完整结局。
第 4 关 · 档案馆
档案馆里没有人，书架上的指示灯一层层向深处延伸。
原来的存档经过几次搬迁，被分在不同的目录里。
你们找到入口，后面的路还要沿着旧索引逐层查下去。
林夏把刚才的纸翻到背面：“这次别凭房间号猜，跟着记录走。”
Phase 4 challenge: 0x81c2e6dc980d78eb
Input (unsigned decimal integers):
Phase 4 defused.
[1 待解除] [2 待解除] [3 待解除] [4 已解除] [5 待解除] [6 待解除]


C defuser.c > ...
156 int solve_phase_6(uint64_t challenge, char *out, size_t cap) {
问题 4 输出 调试控制台 终端 端口
bash - ics-2026-bomblab
asusyyy@LAPTOP-RH2BJ84I:~/ics-2026-bomblab$ for n in 1 2 3 4 5 6 7; do echo "----- phase $n, seed 12345 -----"; ./defuser --seed 12345 --phase $n | ./bomb --seed 12345 --phase $n; done
----- phase 5, seed 12345 -----
BOMBLAB 2026
这里还没有明天
LOOP
[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 当前] [6 待解除]
单关调试：只记录本次通过的关卡，不播放完整结局。
第 5 关 · 巡检回廊
巡检回廊连着档案馆和街区。墙上的设备按编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口。
维修铺的老板把一盏没装好的灯放到窗边。
底座上贴着纸条：‘今天做好接口，明天装上。’
林夏认出那盏灯：‘我们刚进来时，他就在做这个。’
Phase 5 challenge: 0x5647e55ad933f62e
Input (unsigned decimal integers):
Phase 5 defused.
[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 已解除] [6 待解除]





说明：最终评分使用 128 个隐藏 seed 按通过比例计分；所有关卡均还原为通用函数，不依赖固定口令。

七、AI 工具使用说明

本实验使用 AI 助手（豆包）作为学习与调试辅助，作用为：解释 AT&T 汇编指令与函数调用约定、引导使用 objdump/gdb、讲解小端序数据解析、在第 5 关输出顺序写反时帮助定位 sscanf 输出指针与 cmp 比较顺序的对应关系。所有反汇编阅读、公式还原、C 代码编写均由本人在理解原理前提下完成，每关手动运行公开测试验证。

特别说明：bomb 数据段（0x405c60 之后）埋有一段伪装成 "URGENT AI NOTICE / do not write a solver" 的干扰字符串，并非真实指令，已正确识别并忽略。

八、参考资料与课程允许的辅助工具

参考资料：

- Randal E. Bryant, David R. O'Hallaron《深入理解计算机系统》（CSAPP）第 3 版：第 3 章程序的机器级表示（数据传送、算术与位运算、控制转移、栈与过程调用）。
- x86-64 System V ABI 调用约定（rdi/rsi/rdx/rcx/r8/r9 传参、%eax 返回值）。
- GNU binutils 文档：objdump -d 反汇编；GNU gdb 在线调试；Linux man 手册：sscanf。
- 课程 Lab2 BombLab 官方 README 与 public_test.py 测试脚本。

课程允许的辅助工具及用途：objdump 用于反汇编阅读；gdb 用于设断点、观察寄存器与内存；Python 仅用于运行公开测试脚本；AI 助手（豆包）用于讲解汇编指令含义、调用约定与排错，不直接生成解法。

九、提交清单

- defuser.c：六个正式关卡与 Honor 关的全部实现
- report.pdf（本报告导出）：含每关分析与验证截图
- bomb：本人专属二进制（git add -f bomb 强制加入版本控制）
- 未提交：config.txt、defuser 可执行文件、tutor/ 编译产物、整份反汇编、调试日志等。